ChatGPT

Executive Summary

This comprehensive plan lays out a **week-by-week, two-week-per-topic learning roadmap** for a Generative AI (GenAI) developer from Week 15 through Week 52 of 2026. It prioritises advanced GenAI topics that build on your current skills (Python, LLM usage, Azure basics, NLP) and conceptual knowledge of MCP (Model Context Protocol) and Agentic AI. Early topics convert that conceptual knowledge into hands-on experience (e.g. building LLM-powered agents), while later topics deepen and broaden your expertise (e.g. fine-tuning, RAG, multimodal AI, deployment, ethics). Each two-week slot includes clear objectives, prerequisites, activities, deliverables, assessment criteria, time estimates, and authoritative resources with links. We provide a Gantt-style timeline of all topics, a comparative table of difficulty/impact, and biweekly review checkpoints. Finally, we offer a 3-month intensive sprint plan (with detailed weekly/daily tasks) and a lightweight alternative for learners with limited time. This plan uses official documentation (e.g. Microsoft, Anthropic), research papers (e.g. GPT-4, RAG, Chain-of-Thought), and reputable blogs. It is written in en-IN (Indian English) style and aims to be analytical and exhaustive.

```
gantt
    title Generative AI Developer 2026 Learning Plan
    dateFormat  YYYY-MM-DD
    %% Start Monday of week 15 (2026-04-12) as 2026-04-12
    section Topics
    Agentic AI & Tools (MCP Integration)        :a1, 2026-04-12, 14d
    Prompt Engineering & CoT                    :a2, after a1, 14d
    LLM Fine-Tuning & Adaptation                :a3, after a2, 14d
    Retrieval-Augmented Generation (RAG)        :a4, after a3, 14d
    Conversational Agents (Chatbots)            :a5, after a4, 14d
    Multimodal AI (Vision+Language)             :a6, after a5, 14d
    Azure OpenAI & AI Services                 :a7, after a6, 14d
    Deployment & MLOps (Docker, CI/CD)         :a8, after a7, 14d
    Evaluation & Metrics for GenAI             :a9, after a8, 14d
    Advanced Agentic AI (MCP Code Execution)    :a10, after a9, 14d
    Responsible AI & Ethics                    :a11, after a10, 14d
    LLM for Code Generation                    :a12, after a11, 14d
    Model Optimization (LoRA, Quantization)    :a13, after a12, 14d
    Domain-Specific GenAI                      :a14, after a13, 14d
    Data Engineering & Synthetic Data          :a15, after a14, 14d
    LangChain & LLM Frameworks                 :a16, after a15, 14d
    Cutting-edge Research & Trends             :a17, after a16, 14d
    Capstone Project / Portfolio               :a18, after a17, 14d
```

Detailed Two-Week Topic Plans

Agentic AI & Tools (MCP Integration) – Weeks 15-16

Learning Objectives: Understand and apply Agentic AI concepts by building LLM-powered agents that use external tools/APIs. Learn the Model Context Protocol (MCP) standard for connecting LLMs to data and tools [1] [2].
Prerequisites: Python programming, basic familiarity with calling LLM APIs, REST APIs, JSON. Conceptual understanding of Agentic AI and tools (from prior knowledge).

- **Week 1 Activities:**
- **Study MCP & Function Calling:** Read Anthropic's MCP announcement [1] and Google's Apigee blog on MCP [3]. Understand how MCP standardises tool integration.
- **Hands-on Tutorial:** Follow an OpenAI function-calling tutorial (e.g. datacamp or official docs) to call a simple API (weather, calculator) from GPT-4 [2]. Experiment with writing JSON tool descriptions for function calling.

- **Mini-Project:** Build a simple Python agent using LangChain or the OpenAI Python SDK that takes user input and calls a public API (e.g. Wikipedia search or calculator) via function-calling. For example, query the agent for facts and have it search a knowledge API.

- **Week 2 Activities:**

- **MCP Server Practice:** Use Anthropic's MCP SDK (or mock it) to create a local MCP server with at least 2 tools (e.g. one to read local files, one to call a web search). Implement an MCP client in Python to connect to it.
- **Integrate LLM & Tools:** Combine the tools into an agent: e.g. ask your agent a question that requires reading a document or searching the web. Verify the agent uses the MCP tools rather than static knowledge.
- **Iteration & Reflection:** Optimize prompts so the agent reliably calls tools (experiment with chain-of-thought or direct instructions).

Deliverables: A working Python agent (script or notebook) that uses an LLM (GPT-4/Claude) together with at least one external tool (via function-call or MCP). Submit the code and a short report on how tools were integrated.
Assessment Criteria: Agent correctly calls tools based on input prompts; integration is modular (e.g. tools separate from prompt). Evaluate by demoing at least 2 use cases. Check understanding by explaining how MCP simplifies adding new tools [1].

Time Commitment: ~30–40 hours (15-20h/week).
Recommended Resources: Official OpenAI Function Calling docs; Anthropic MCP overview [1]; Google Apigee blog on MCP [3]; LangChain Agents Guide (official docs) [4] [5].
Tools: OpenAI or Anthropic API (Claude) via SDK, Python (requests), LangChain or AutoGPT framework, MCP SDK or Flask for server, Postman (optional).

Checkpoint: End of Week 16 – Verify agent uses tools (not hardcoded answers). If agent fails on edge cases (e.g. ambiguous queries), refine prompts or add more tool definitions. Confirm ability to add a new tool to the MCP server with minimal changes.

Advanced Prompt Engineering & Chain-of-Thought (CoT) – Weeks 17-18

Learning Objectives: Master advanced prompting techniques, including few-shot, chain-of-thought and multi-turn prompts, to improve LLM reasoning [6]. Learn to structure prompts for complex tasks (summarization, logic puzzles, arithmetic).
Prerequisites: Comfortable with Python and using chat/completion APIs; basic prompt-writing skills.

- **Week 1 Activities:**
- **Study CoT Concept:** Read the Chain-of-Thought paper [6] (Wei et al. 2022) to understand how showing intermediate reasoning steps can boost LLM performance. Watch tutorials or blog summaries on CoT prompting.
- **Prompt Strategy Practice:** Take a few sample tasks (math word problems, common-sense QA) and design few-shot examples with and without CoT explanations. Use GPT-4 or Claude to test performance. Compare accuracy improvements qualitatively (CoT vs direct answer).
- **Experiment:** Try self-consistency (multiple CoT examples) and ensemble prompting methods. Document which strategies help most on which tasks.
- **Week 2 Activities:**
- **Complex Prompting:** Tackle a complex NLP task (e.g. document summarization, reading comprehension). Craft prompts that guide the model through multiple steps (e.g. first extract facts, then summarize).
- **Prompt Templates & Tools:** Learn about prompt templates (LangChain, PromptLib). Implement a simple prompt template in code (Python/Prometheus) to inject dynamic content.
- **Refinement & Benchmarks:** Evaluate your prompts on a validation set or known answers. Iterate prompts to reduce hallucination and improve relevance.

Deliverables: A set of 3–5 prompt templates (code + descriptions) demonstrating advanced techniques (few-shot, CoT) and results on sample tasks (e.g. before/after accuracy or output examples). A brief write-up comparing performance.
Assessment Criteria: Ability to significantly improve task accuracy or output quality using CoT techniques [6]. Clear documentation of prompt designs. Peer review or self-critique on prompt clarity and reliability.

Time Commitment: ~20–30 hours.
Recommended Resources: Chain-of-Thought paper [6]; OpenAI Cookbook examples on prompting; OpenAI docs or blogs on prompt best practices; LangChain PromptGuide.
Tools: GPT-4/Claude via API, Python, Jupyter/Colab, LangChain.

Checkpoint: End of Week 18 – Verify prompts improve performance (compare baseline vs advanced prompting). If not, try different CoT examples or adjust prompt phrasing. Confirm understanding by explaining why CoT helps in your cases.

LLM Fine-Tuning & Adaptation – Weeks 19-20

Learning Objectives: Learn how to fine-tune a pretrained LLM on domain-specific data (SFT/DPO), including techniques like Low-Rank Adaptation (LoRA) to efficiently adapt models [7]. Compare fine-tuning vs in-context learning trade-offs.
Prerequisites: Familiarity with machine learning training concepts, basics of PyTorch/Transformers, having practiced using an LLM API.

- **Week 1 Activities:**
- **Concepts & Techniques:** Read Microsoft's guide on fine-tuning (Azure Foundry) [8] [7]. Note benefits: higher quality outputs than prompt-only, token and latency savings. Understand LoRA basics (train a small subset of weights).
- **Dataset Prep:** Select or create a small dataset (e.g. customer support Q&A, medical Q&A, or code examples) in JSONL format as required. Follow Azure's sample JSONL format [9] for chat fine-tuning.
- **Initial Experiment:** Use Hugging Face Transformers `Trainer` (or Azure CLI/Foundry if accessible) to fine-tune a smaller model (e.g. Llama-2-7B or GPT-Neo) on your dataset. Monitor training, verify no major errors.
- **Week 2 Activities:**
- **LoRA & Efficiency:** Implement LoRA-based fine-tuning on the same data (HF PEFT library). Compare training time and resource usage vs full fine-tune.
- **Evaluate Fine-Tuned Model:** Use held-out examples to compare responses of base vs fine-tuned model. Check for improved relevance or correctness.
- **AutoML Tools (optional):** Explore any Azure ML automated tools for dataset creation or labeling to augment your data.

Deliverables: A fine-tuned (and optionally LoRA-adapted) LLM on your dataset. Submit training configuration/code and sample outputs before/after. Include evaluation (e.g. simple accuracy or human review notes).
Assessment Criteria: Fine-tuned model should show measurable improvement (e.g. higher task accuracy or BLEU/ROUGE on test set). Document how LoRA reduced compute/cost [7]. Ensure understanding of when to use fine-tuning over prompting.

Time Commitment: ~30–40 hours.
Recommended Resources: Microsoft Fine-Tuning guide [8]; Hugging Face Transformers Fine-tuning docs [10]; LoRA/PEFT tutorials (HF docs).
Tools: Hugging Face Transformers and Datasets, Azure OpenAI or Azure ML (Foundry) if available, Python, GPU instance (Colab/GPU).

Checkpoint: End of Week 20 – Confirm fine-tuned model produces higher-quality outputs on test prompts [8]. If not, refine dataset (add examples) or training hyperparameters. Check that LoRA model does not degrade performance significantly while reducing cost [7].

Retrieval-Augmented Generation (RAG) – Weeks 21-22

Learning Objectives: Implement RAG pipelines that combine LLMs with an external knowledge base via embeddings. Learn how dense retrieval (vector search) can improve factuality [11] [12].
Prerequisites: Knowledge of embeddings, familiarity with vector stores (basic idea), comfort with LLM APIs.

- **Week 1 Activities:**
- **Learn RAG Concept:** Read the RAG paper [11] (Lewis et al. 2021) to understand how LLMs integrate parametric (model) and non-parametric (retrieval) memory. Focus on the idea of fine-tuning a seq2seq model with a retriever.
- **Implement Basic RAG:** Use Hugging Face's `RagTokenizer` / `RagRetriever` pipeline or LangChain LLM+retriever combo. For example, index a corpus of text (Wikipedia snippets or your own articles) with FAISS or Milvus, then query via GPT.
- **Test Q&A:** Evaluate the RAG system on open-domain questions. Compare answers from RAG vs a vanilla LLM.
- **Week 2 Activities:**
- **Azure Cognitive Search + LLM:** Set up an Azure Cognitive Search vector index (using Azure AI Search) [12]. Ingest documents (Azure's Quickstart guides). Use Azure OpenAI to query this index (using integrated vector queries as described).
- **Hybrid Search Experiments:** Try hybrid queries (keyword + vector) for more robust results [13]. Build a simple chatbot interface that retrieves docs and then asks the LLM to generate an answer based on retrieved context.
- **Iterate on Retrieval:** Tune the embedding model or index parameters (e.g. k-nearest neighbors count) for best precision.

Deliverables: A RAG-based QA system. Provide code/notebook for indexing documents and querying, plus example Q&A interactions showing retrieved context. Include a brief report of performance (e.g. precision@k or user satisfaction).
Assessment Criteria: RAG should demonstrably provide more factual, detailed answers than a base LLM alone. Cite the RAG paper to explain your approach [11]. Ensure retrieval hits relevant passages for queries.

Time Commitment: ~30 hours.
Recommended Resources: RAG research paper [11]; Hugging Face RAG tutorial; Azure Vector Search overview [12]; Microsoft AI Search docs on embeddings and hybrid search [13].
Tools: Hugging Face Transformers, FAISS or Pinecone, Azure AI Search or Elasticsearch with vector plugin, Python, LangChain (optional).

Checkpoint: End of Week 22 – Check that at least 70–80% of queries retrieve relevant doc excerpts. If not, refine document corpus or adjust embeddings (e.g. use OpenAI embeddings). Confirm RAG answers are more accurate/factual than LLM alone.

Conversational Agents & Chatbots – Weeks 23-24

Learning Objectives: Build interactive chatbots using LLMs. Learn dialogue management fundamentals and how to integrate LLMs into conversational flows (slot-filling, context memory).
Prerequisites: Basic NLP knowledge, LLM usage, and experience from previous CoT and RAG topics.

- **Week 1 Activities:**
- **Study Conversational AI:** Review chatbot architectures (Retrieval vs Generative). Read Microsoft's guide or blog on building chatbots with Azure OpenAI [14]. Explore Rasa or Bot Framework basics.
- **Prototype Bot:** Create a simple rule-based or retrieval-based chatbot (no LLM) on a toy topic (e.g. FAQ). Familiarise with Azure Bot Service or Rasa for conversation flows.
- **Integrate LLM:** Modify the bot to call an LLM for generating responses. For example, use OpenAI API as the backend of a Rasa action or Bot Framework "skill".
- **Week 2 Activities:**
- **Multi-turn Context:** Implement conversation history tracking. Ensure the bot passes previous messages or summaries to the LLM so it maintains context over turns.
- **Persona and Constraints:** Experiment with system prompts or LLM roles to give the bot a consistent persona or tone. (E.g. "You are a helpful travel assistant.")
- **User Inputs & Data Sanitization:** Build logic for handling unclear questions (re-prompt user) and ensure safe handling of inputs (filter PII).

Deliverables: A demo conversational agent (web interface or chat log). Must support a coherent multi-turn dialogue on a specific domain (e.g. tech support or tutoring). Provide instructions to run it and example interactions.
Assessment Criteria: Evaluate on coherence, relevancy, and context retention across turns. Use a small user study or self-test (5–10 conversations) to score the bot's helpfulness. Document any challenges (like hallucinations or repetitive answers) and how you mitigated them.

Time Commitment: ~20–30 hours.
Recommended Resources: Azure Bot Service with OpenAI example [15]; Rasa documentation on integrating LLMs; Microsoft QnA Maker (for quick retrieval-based baseline).
Tools: Azure Bot Service or Rasa, Azure OpenAI/GPT API, Python/JavaScript, possibly low-code tools (Power Virtual Agents).

Checkpoint: End of Week 24 – Verify the bot correctly maintains context (e.g. refers back to earlier user details). If it fails, adjust how history is passed (e.g. summary vs full log). Ensure polite and helpful responses.

Multimodal AI (Vision + Language) – Weeks 25-26

Learning Objectives: Explore multimodal generative models. Learn how language models can process images (e.g. GPT-4 Vision) and how vision-language models like CLIP work. Build an application combining

vision and text.
Prerequisites: Familiarity with LLMs; basic image processing.

- **Week 1 Activities:**
- **Study Multimodal Models:** Read OpenAI's GPT-4 Technical Report [16] [17] focusing on vision capabilities (GPT-4V). Explore CLIP (Contrastive Language-Image Pretraining) idea and DALL·E/Stable Diffusion basics.
- **Hands-on CLIP:** Use a pretrained CLIP model (via Hugging Face Transformers or OpenAI API) to encode images and text, and perform similarity tasks (e.g. rank which caption best matches an image).
- **Simple Vision-Language Task:** Choose an existing image dataset (e.g. COCO or your photos). Write a script that takes an image and generates a caption using a multimodal LLM (if available) or an image-to-text pipeline.
- **Week 2 Activities:**
- **Vision+Chatbot App:** Build a small app where a user can upload an image and chat about it. Use GPT-4V (if accessible) or combine CLIP embeddings + GPT: e.g. retrieve similar image captions from dataset and feed them to GPT for Q&A.
- **Image Generation (optional):** Experiment with stable diffusion or DALL·E: e.g. write a program that generates images from textual descriptions. Evaluate creativity vs accuracy.
- **Analysis:** Compare outputs from different multimodal approaches. Note limitations (e.g. GPU needs, quality).

Deliverables: A multimodal demo (e.g. web app or script) showing image + text processing. For example, an image-caption bot or an image-to-text Q&A assistant. Include sample images and interactions.
Assessment Criteria: Demonstrate that the model correctly understands simple image content (e.g. "What objects are in this image?" answered accurately). If generating images, ensure they match prompts. Discuss any failure modes. Cite GPT-4's multimodal capability [16] as context.

Time Commitment: ~20 hours.
Recommended Resources: GPT-4 Technical Report [16]; OpenAI GPT-4V documentation; Hugging Face CLIP tutorial; Stable Diffusion online resources (Hugging Face Spaces).
Tools: OpenAI API (GPT-4V or API functions), Hugging Face Transformers (CLIP), Stable Diffusion (Diffusers), Python, Streamlit/Gradio for UI.

Checkpoint: End of Week 26 – Check that the application correctly handles both text and image inputs. If misclassification occurs, consider adding more prompts or using better pre-processing (e.g. resizing, cropping). Confirm app stability with several test images.

Azure GenAI Services & Cloud Integration – Weeks 27-28

Learning Objectives: Master Azure's Generative AI offerings. Learn to deploy LLM applications using Azure OpenAI Service, Azure Cognitive Search, and other AI services (e.g. Azure AI Studio). Integrate cloud APIs

securely.
Prerequisites: Basic Azure knowledge (resource groups, roles), completion of previous LLM and RAG topics.

- **Week 1 Activities:**
- **Azure OpenAI Setup:** Follow Microsoft Learn tutorials to provision an Azure OpenAI resource (or Foundry) and create a simple "chat completion" through Azure CLI or portal. Review Microsoft's generative AI docs [8].
- **Cognitive Services:** Explore Azure AI Search (indexing text with AI enrichment) and QnA Maker. Try uploading a document set to Azure Cognitive Search and querying it.
- **Security & Roles:** Understand Azure roles required (e.g. AI Owner vs User [18]) and practice setting permissions.
- **Week 2 Activities:**
- **Azure-based Project:** Build a small Azure-hosted GenAI app. Example: an Azure Function or Web App that takes user input, calls Azure OpenAI for answer, and optionally uses Cognitive Search for RAG. Deploy and test it.
- **Cost & Monitoring:** Set up Azure Monitor or Logging to track usage/cost of your GenAI resource. Practice scaling (e.g. replica instances, region selection).
- **Tooling:** Try using Azure AI Studio (or Foundry UI) for pipeline building or data preparation.

Deliverables: A deployed Azure-hosted LLM application (demo URL or local host). Submit architecture diagram and key configurations. Include any ARM templates or pipeline scripts used.
Assessment Criteria: The app should correctly call Azure GenAI services and handle errors gracefully (rate limits, auth issues). Evaluate scalability (could it handle 100 concurrent requests?). Demonstrate cost-awareness (use lower-priced model if possible, or have alerts).

Time Commitment: ~20 hours.
Recommended Resources: Azure OpenAI and Cognitive Services docs (e.g. Microsoft Learn "Develop generative AI with Azure"); Microsoft Foundry fine-tune guide [8]; Azure AI Search Quickstart [12].
Tools: Azure Portal, Azure CLI, Azure AI Studio (Foundry), Visual Studio Code, Azure ML studio or Databricks (optional).

Checkpoint: End of Week 28 – Verify your app works on the Azure cloud (not just locally). Check that logs indicate successful API calls and no permission errors. If issues, revisit role assignments or network settings (NSG, firewalls).

Deployment & MLOps for GenAI – Weeks 29-30

Learning Objectives: Learn best practices for deploying GenAI applications into production. Master containerization (Docker), CI/CD pipelines, and monitoring/traceability (MLflow, LangSmith, Azure ML).
Prerequisites: General DevOps knowledge (Git, Docker), and all previous LLM project experience.

- **Week 1 Activities:**

- **Containerization:** Dockerize one of your LLM applications (e.g. the chatbot or RAG app). Write a `Dockerfile` and build an image with the LLM API client. Test container locally.
- **Version Control:** Ensure code is on Git/GitHub. Practice branching and PR reviews for any code changes.

- **CI Pipeline:** Set up a simple CI pipeline (GitHub Actions or Azure Pipelines) that automatically builds and tests the Docker image on each commit.

- **Week 2 Activities:**

- **Deployment:** Deploy the container to a cloud platform. For example, use Azure Container Instances or Azure Kubernetes Service (AKS) for scalability. Configure environment variables (API keys) securely.
- **Monitoring & Logging:** Integrate a logging framework (e.g. Azure Application Insights or MLflow tracking) to record query inputs/outputs. Set up alerts for errors or high latency.
- **Model Registry (optional):** Explore using MLflow or Hugging Face Hub to version your fine-tuned models.

Deliverables: A fully deployed GenAI app (e.g. in AKS) with CI/CD pipeline. Provide a README on how to build/deploy from source. Include screenshots of CI runs, container image, and monitoring dashboard.
Assessment Criteria: The app should spin up in a few minutes via your pipeline. Validate that it recovers from crashes (e.g. restarts). Show logs of an LLM query in Application Insights. Include a rollback test (e.g. revert a commit, pipeline does not deploy a bug).

Time Commitment: ~30 hours.
Recommended Resources: Azure DevOps / GitHub Actions docs; Docker official docs; Azure AKS tutorial; MLflow or LangSmith docs for model monitoring.
Tools: Docker, Kubernetes/AKS or Azure Container Apps, GitHub Actions, Azure Monitor/Application Insights, MLflow.

Checkpoint: End of Week 30 – Trigger the pipeline with a sample commit and ensure successful deployment. Confirm you can view a log entry in monitoring when making a query. If not working, check pipeline logs and network settings.

Evaluation & Metrics for Generative AI – Weeks 31-32

Learning Objectives: Learn how to evaluate generative models: metrics for accuracy, fluency, and safety. Understand evaluation methods (human reviews, automated metrics like BLEU/ROUGE, or specialized safety tools). Learn to track bias and performance drift.
Prerequisites: Experience building generative outputs, basic stats knowledge.

- **Week 1 Activities:**
- **Automated Metrics:** Study standard metrics: BLEU, ROUGE, METEOR for text; perplexity. Use these to evaluate your fine-tuned model vs baseline on a test set (e.g. some QA or summarization).
- **Factuality & Hallucinations:** Research approaches for measuring factual accuracy (e.g. QAGen benchmarks). Try an LLM evaluator like GPT-4 as a judge (LLM eval).

- **Fairness & Bias:** Review literature on AI fairness metrics. Run a fairness audit: e.g. feed your bot prompts with different gender/race contexts and compare responses.

- **Week 2 Activities:**

- **User Feedback Loop:** Design a simple feedback mechanism (e.g. thumbs up/down UI) in one of your apps. Collect sample "user" feedback on answers for analysis.
- **Safety Filters:** Experiment with OpenAI Moderation API or Azure Content Moderator on your outputs. Check for banned content triggers.
- **Documentation:** Prepare an evaluation report summarizing metrics and any discovered issues.

Deliverables: An evaluation summary document. Include metric scores (tables or charts) for your models, examples of successes/failures, and steps taken to mitigate problems. If feasible, include a user feedback log.
Assessment Criteria: Show clear evidence of rigorous testing: e.g. X% accuracy on dataset, or detection of hallucinations. Discuss any biases found and how to address them (citing guidelines). The report should convincingly demonstrate model readiness.

Time Commitment: ~20 hours.
Recommended Resources: Papers on LLM evaluation (e.g. ChatEval); tutorials on BLEU/ROUGE; Microsoft Responsible AI Toolkit; OpenAI Moderation API docs.
Tools: Python (NLTK/ROUGE libraries), Jupyter, any dashboards (e.g. MLflow for metrics), OpenAI/Anthropic eval models.

Checkpoint: End of Week 32 – Ensure at least one automated metric and one human-eval method are documented. If scores are low, iterate on model or data. Confirm any content filters flag inappropriate outputs.

Advanced Agentic AI (Code Execution with MCP) – Weeks 33-34

Learning Objectives: Deepen agentic AI by enabling agents to write and execute code (programmatic tool use) as in Anthropic's MCP code execution model [2]. Build agents that can dynamically load tools (APIs/code) and perform complex tasks.
Prerequisites: Python coding skills, experience with previous agent projects, MCP basics.

- **Week 1 Activities:**
- **Study Code Execution:** Read Anthropic's "Code execution with MCP" blog [2]. Understand how loading tools as code (file APIs) reduces context usage and costs.
- **Build Code-Agent Skeleton:** Create a Python agent framework where tools are implemented as Python functions (instead of on-model calls). E.g., have a folder of `.py` scripts each defining a tool.

- **Tool Discovery:** Implement a mechanism for the agent to list available tool scripts (e.g. using `os.listdir`) and import them on-demand.

- **Week 2 Activities:**

- **Agent Script Writing:** Give the LLM a task requiring multiple steps (like data extraction followed by analysis). The LLM should output code (JavaScript/Python) that calls the needed tools. Execute this code in a secure subprocess.
- **Efficiency Gains:** Measure token usage for tool descriptions in-context vs via code execution. Demonstrate lowered token footprint as in Anthropic's example.
- **Error Handling:** Implement timeouts and sanitization when running LLM-generated code to avoid malicious loops.

Deliverables: A proof-of-concept "code agent": e.g. ask it to take data from one API and push to another via generated code. Include logs showing tool discovery and code execution steps. Provide documentation of the tool directory structure.
Assessment Criteria: Agent successfully writes and runs code to achieve tasks without human intervention. Compare token usage with and without code mode. Cite Anthropic's approach [2] to explain your design.

Time Commitment: ~30 hours.
Recommended Resources: Anthropic Code Execution blog [2]; Python's `subprocess` documentation; LangChain "Agents with Python" examples; security best practices for code execution.
Tools: Python, MCP SDK or custom micro-framework, Docker for sandboxing (optional), GPT-4/Claude API.

Checkpoint: End of Week 34 – Verify the agent can call at least 3 tools via generated code and handle tool output correctly. Check for infinite loop protections. If it fails, refine the prompt that instructs LLM to write correct code (show example code).

Responsible AI & Ethics in GenAI – Weeks 35-36

Learning Objectives: Understand the societal and ethical implications of GenAI. Learn best practices for fairness, privacy, transparency, and compliance (e.g. GDPR, data privacy). Study principles (OECD, EU) for trustworthy AI.
Prerequisites: Awareness of AI ethics issues (general), knowledge of legal/regulatory basics.

- **Week 1 Activities:**
- **Ethical Frameworks:** Read OECD AI Principles or IEEE Trustworthy AI guidelines. Summarize key points (e.g. human-centric, transparent AI).
- **Bias Mitigation:** Research tools/methods for mitigating bias in LLMs (e.g. data balancing, adversarial filtering). If time permits, run the Inclusive Images or similar bias tests on your data.
- **Privacy & Security:** Study privacy laws relevant to AI (GDPR data minimization, user consent). Plan how user data in your applications should be handled (encryption, anonymization).
- **Week 2 Activities:**
- **AI Policy (Org):** Draft a basic "AI use policy" for your projects: include data handling, review processes for outputs, user disclaimers.
- **Case Study:** Analyze a recent GenAI incident (e.g. a widely-reported hallucination or bias case) and write a short memo on lessons learned.

- **Discussion:** If possible, discuss with peers or mentors about responsible deployment: when not to allow certain content, etc.

Deliverables: A brief report or presentation on Responsible AI. Include (a) a summary of standards/guidelines followed; (b) an audit of one of your apps for bias or privacy gaps; (c) an action plan to address issues (e.g. add filters, provide transparency to users).
Assessment Criteria: Show clear understanding of principles like fairness and privacy. Action plans should be concrete (e.g. implement differential privacy or set up red-team reviews). Even if speculative, demonstrate ethical reasoning.

Time Commitment: ~15 hours.
Recommended Resources: OECD AI Principles; EU AI Act summary; Articles on LLM hallucination/bias; OpenAI's safety docs [19]; IEEE Ethically Aligned Design.
Tools: No special tools, but use bias detection libraries (e.g. IBM AI Fairness 360) if desired.

Checkpoint: End of Week 36 – Ensure you have identified at least two ethical risks in your work (e.g. potential user harm, privacy leaks). If any remain unaddressed, plan mitigations before going live.

LLM for Code Generation – Weeks 37-38

Learning Objectives: Leverage LLMs to assist coding. Explore Codex/GPT's coding capabilities. Build tools (e.g. a coding assistant or test generator) that use LLMs to write or analyze code.
Prerequisites: Strong Python skills, familiarity with LLM APIs, and prior agent/prompt experience.

- **Week 1 Activities:**
- **Study Codex/Coding LLMs:** Read about OpenAI Codex / GPT-4 Code (system card) to understand capabilities and limitations. Look at examples of GPT writing functions.
- **Hands-on:** Use OpenAI Codex (via API) to solve a few programming tasks (e.g. "write a function that sorts a list of names ignoring case"). Compare with vanilla GPT-4.

- **Unit Test Generation:** Experiment by asking the model to generate unit tests for a simple function (code->tests).

- **Week 2 Activities:**

- **Develop a Coding Assistant:** Create a simple chat interface (could be CLI) where a user can ask coding questions (e.g. "How to merge two DataFrames in Pandas?") and the LLM responds with code. Implement code execution sandbox to test the snippets (careful with security).
- **Analyze Code:** Ask the LLM to explain or refactor a piece of code you provide. Evaluate answer correctness.
- **Performance Tuning:** Try different prompt styles: fully direct code generation vs step-by-step explanation.

Deliverables: A working coding assistant tool (script or small web app). Include sample interactions: e.g. user query, LLM code output, execution result. Discuss areas where the LLM succeeded or failed.
Assessment Criteria: Assistant should handle at least 5 distinct coding queries correctly. Code samples

should run without syntax errors and meet specs. Evaluate on clarity and correctness of code. Cite OpenAI docs or Codex paper for context.

Time Commitment: ~20 hours.
Recommended Resources: OpenAI "Codex" docs; GPT-4 Code system card; GitHub Copilot documentation; StackOverflow Developer Surveys (for example questions).
Tools: OpenAI Codex/GPT-4 via API, Python, possibly a REPL/sandbox environment (e.g. Jupyter), Flask or Streamlit for interface.

Checkpoint: End of Week 38 – Check that the assistant's code runs correctly for test queries. If it produces bugs, refine prompts or add examples. Confirm understanding by manually reviewing LLM answers for logic errors.

Model Optimization & Efficiency (LoRA, Quantization) – Weeks 39-40

Learning Objectives: Learn techniques to make LLMs efficient: quantization (lower precision) and distillation/LoRA. Study how to deploy smaller models for cost savings [7].
Prerequisites: LLM fine-tuning experience, knowledge of model internals.

- **Week 1 Activities:**
- **Quantization Theory:** Read about quantization techniques (8-bit, INT4). Use Hugging Face's `transformers` and `bitsandbytes` to quantize a small LLM (e.g. quantize a GPT-J model to 4-bit).
- **LoRA Deep Dive:** If not already done, review PEFT/LoRA docs. Try varying LoRA rank to see trade-offs.
- **Speed Benchmarking:** Measure inference speed/memory before and after optimization on small hardware (use `perf` or `time`).
- **Week 2 Activities:**
- **Distillation:** Optionally, implement model distillation (e.g. train a smaller model on outputs of a large model). Use Hugging Face's distillation examples if available.
- **Apply to App:** Replace your LLM in one of the earlier demos (e.g. chatbot) with the optimized version. Check if performance is still acceptable.
- **Cost Analysis:** Compute estimated cost savings for running optimized model at scale.

Deliverables: A report or notebook showing optimization steps. Include performance metrics (latency, memory) before/after. Demonstrate a working quantized/LoRA model in your application.
Assessment Criteria: The optimized model should have notably lower resource use (e.g. >50% speedup or memory reduction) with minimal drop in accuracy. Include logs or tables of resource metrics. Cite the Azure fine-tuning doc on LoRA benefits [7].

Time Commitment: ~25 hours.
Recommended Resources: Hugging Face documentation on quantization and PEFT; Papers on quantization (LLM.int8 or QLoRA); Microsoft docs [7].

Tools: Hugging Face Transformers, bitsandbytes library, PyTorch, Python, hardware (local or Colab with GPU).

Checkpoint: End of Week 40 – Confirm quantized/LoRA model still produces coherent outputs. If accuracy suffers too much, adjust quantization bit-width or LoRA rank. Ensure that improvements meet your speed/memory goals.

Domain-Specific GenAI Applications – Weeks 41-42

Learning Objectives: Adapt generative AI to specialized domains (e.g. medical, legal, finance). Learn domain fine-tuning and specialized retrieval. Understand domain compliance (e.g. HIPAA for health).
Prerequisites: Completion of fine-tuning/RAG topics, plus domain knowledge or willingness to research.

- **Week 1 Activities:**
- **Choose Domain:** Select a domain of interest (e.g. healthcare). Gather domain-specific corpus (e.g. PubMed abstracts, legal documents).
- **Custom Data Prep:** Clean and preprocess domain text for finetuning or RAG. Ensure formatting preserves confidentiality if needed (anonymize data).
- **Baseline Run:** Evaluate your general LLM on a small set of domain queries (e.g. clinical questions) to identify gaps.
- **Week 2 Activities:**
- **Apply RAG or Fine-Tune:** Depending on data volume, either fine-tune the LLM on domain data or build a specialized RAG index using domain texts.
- **Evaluation:** Test the domain-adapted model vs base on a benchmark (if available) or synthetic queries. Measure improvement in domain accuracy.
- **Compliance Considerations:** Document any regulatory requirements (e.g. patient data privacy) and ensure your process respects them (only public data, etc.).

Deliverables: A domain-specific GenAI demo (e.g. a Q&A on medical facts). Include a short technical note on how you adapted the model/data, and any domain-specific evaluation results.
Assessment Criteria: The domain-adapted model should clearly outperform the base model on domain queries (demonstrated via examples or metrics). Confirm any domain restrictions were handled (e.g. no PHI leaks).

Time Commitment: ~20 hours.
Recommended Resources: Domain corpora (e.g. PubMed, arXiv), relevant regulations (HIPAA summary if medical); Tutorials on customizing LLMs for domain use (e.g. FinBERT for finance).
Tools: Hugging Face, domain-specific APIs (optional), Python.

Checkpoint: End of Week 42 – Ensure your model's outputs are accurate for domain queries. Check that no out-of-domain hallucinations occur (e.g. referencing non-existent studies). If issues, refine data or use domain expert input.

Data Engineering & Synthetic Data for GenAI – Weeks 43-44

Learning Objectives: Learn to prepare high-quality training data for GenAI. Explore synthetic data generation to expand datasets. Understand data pipelines (scraping, cleaning, annotation) for building corpora.
Prerequisites: Python scripting for data processing, knowledge of previous fine-tuning steps.

- **Week 1 Activities:**
- **Data Collection:** Pick a topic and scrape or gather relevant text data (e.g. web APIs, public datasets). Use Python (BeautifulSoup, Selenium) or APIs (Reddit, news).
- **Cleaning & Annotation:** Clean the data (remove HTML, duplicates). If building a dataset for a task (like QA), annotate a small subset manually (or using crowdsourced guidelines).
- **Pipeline Tools:** Familiarize with data pipelines (e.g. using Pandas, Apache Airflow, or Azure Data Factory for larger flow). Build a script that processes raw data to a clean JSONL.
- **Week 2 Activities:**
- **Synthetic Data Generation:** Use your LLM to augment data. For instance, generate paraphrases or new QA pairs. Validate quality by manual review.
- **Quality Checks:** Write tests (regex or scripts) to check dataset integrity (no nulls, proper JSONL format, token length within limits).
- **Reuse & Versioning:** Store dataset in a version control (Git LFS or DVC) and document schema (e.g. column meanings).

Deliverables: A packaged dataset ready for fine-tuning or evaluation. Include data pipeline code and a README describing source, cleaning steps, and format. Optionally, include synthetic vs real data comparison results.
Assessment Criteria: The dataset should be non-trivial in size (e.g. 1000+ examples). Check that it's clean (no formatting errors) and relevant. Demonstrate understanding by explaining one data challenge and how you solved it (e.g. handling outliers).

Time Commitment: ~15 hours.
Recommended Resources: Articles on data preparation for ML; Azure Data Factory tutorials; Hugging Face Datasets guide; Cloud provider data lakes (if applicable).
Tools: Python (pandas, requests, BeautifulSoup), DVC/Git for data versioning, Azure Data Factory or Apache Airflow (optional for pipeline orchestration).

Checkpoint: End of Week 44 – Validate dataset with a small model test (e.g. see if a simple fine-tune yields non-zero improvements). If data quality issues arise, iterate on cleaning.

LangChain & LLM Frameworks – Weeks 45-46

Learning Objectives: Gain proficiency with popular GenAI developer frameworks. Use LangChain (chains, agents, memory) and LlamaIndex/RAG frameworks to build complex apps more efficiently.

Prerequisites: Completed agent and RAG topics; Python experience; this assumes motivation to use a higher-level library.

- **Week 1 Activities:**
- **LangChain Basics:** Follow LangChain docs to build a simple chain: e.g. a conversation chain or LLMChain that formats a prompt with user input [20].
- **Memory & State:** Learn LangChain memory modules. Implement a chatbot with memory where it remembers user's name/preferences across turns.
- **LlamaIndex (or Alternative):** Explore LlamaIndex (GPT Index) for knowledge retrieval. Index a custom document (e.g. your notes) and query it via LLM.
- **Week 2 Activities:**
- **Multi-Agent Systems:** If ambitious, experiment with LangChain's multi-agent systems or LangGraph to coordinate multiple agents (each with a specialty). For example, combine a retrieval agent and a summarization agent.
- **Debugger Tools:** Use LangSmith (or any available tracing tool) to debug chain execution. Analyze token usage and errors in chain steps.
- **OpenAI Agents SDK:** Optionally try using OpenAI's new Agents SDK (if accessible) to compare with LangChain.

Deliverables: A mini-project built entirely with LangChain (and optionally LlamaIndex). For instance, a multi-step assistant that uses multiple chains or tools (search, database lookup). Provide code and a flowchart of the chain.
Assessment Criteria: Code should use the framework idiomatically (e.g. using `LLMChain`, `AgentExecutor`). The app should run correctly, and a log/tracer should show each chain step. Cite LangChain documentation or SDK references to explain structure.

Time Commitment: ~25 hours.
Recommended Resources: LangChain official docs (Chains, Agents) [20]; LangChain YouTube tutorials; LlamaIndex documentation; OpenAI Agents SDK docs.
Tools: LangChain (Python), LlamaIndex or equivalent, Python, LangSmith (for tracing).

Checkpoint: End of Week 46 – Ensure your LangChain app handles a complete user request without error. If a component fails (e.g. retrieval returns empty), debug with provided tools. Confirm you understand each part of the chain by explaining it.

Cutting-edge Research & Trends – Weeks 47-48

Learning Objectives: Survey the latest GenAI research to stay ahead. Topics may include RLHF advancements, emerging model architectures, open-source LLM progress, or novel applications (e.g. AI agents in software development).
Prerequisites: Broad GenAI knowledge (as above). Openness to reading research.

- **Week 1 Activities:**

- **Read Recent Papers:** Pick 2–3 high-impact recent papers (2024–2025) in GenAI. Possibilities: "OpenAI Evals", "GPT agents evaluation", "LLM alignment survey". Summarize key findings.
- **Open Source Models:** Explore new open models (e.g. LLaMA-3, Mistral-7B). Read their release notes or paper abstracts. Evaluate if/how you could use them.
- **Community Tools:** Try a new tool or library you haven't used (e.g. Ray's RL for LLMs, a new annotation tool, or an agent playground).
- **Week 2 Activities:**
- **Apply a Research Idea:** Take one promising idea (e.g. Alpaca or RLHF methods) and implement a toy version. For example, fine-tune a model on human preference data, or apply a new decoding strategy (like Self-Consistency decoding).
- **Benchmark:** If possible, evaluate your implementation against a baseline on a public benchmark (e.g. HuggingFace leaderboard or a small metric test).
- **Presentation:** Prepare a short presentation (slides or document) on "Future of GenAI" focusing on what you learned.

Deliverables: A research review document (with references) and any prototype code. For example, include a summary of one paper and some illustrative results from your mini-experiment.
Assessment Criteria: Demonstrate critical understanding of the research. In your prototype, clearly state what was replicated and what the outcome was. Even a negative result is fine if well-explained.

Time Commitment: ~20 hours.
Recommended Resources: ArXiv.org (2024-25 GenAI papers); Hugging Face Model hub (for open models); AI research blogs (Anthropic, DeepMind, etc.); GPT-Engineer or Oobabooga new tools.
Tools: Python, Pytorch (for experiments), any necessary LLM API (if using GPT for eval), Google Scholar.

Checkpoint: End of Week 48 – Verify that your write-up captures at least 3 new trends. If a prototype was built, ensure it runs and connects to a baseline. Discuss in writing how it advances or differs from known approaches.

Capstone Integration Project – Weeks 49-50 (Bonus Weeks)

Learning Objectives: Consolidate all learning in a final project. Build a full-stack GenAI application or research deliverable that showcases multiple skills (prompting, agentic LLM, cloud deployment, etc.).
Prerequisites: Completion of most prior topics; ability to plan independently.

- **Weeks 49-50 Activities:**
- **Project Planning:** Define the project (e.g. an AI assistant for a specific domain, or an end-to-end GenAI pipeline). Outline features drawing on earlier modules (e.g. uses RAG, has UI, deployed on Azure).
- **Implementation:** Execute the plan. This is largely self-directed: use code from previous work, integrate services, refine UI.
- **Testing & Documentation:** Thoroughly test all components. Prepare a report or presentation summarizing the project architecture, chosen techniques, and results.

Deliverables: A deployed end-to-end GenAI application or a detailed project report (with code link). Could be a video demo or a live web app. Include discussion of how prior topics were integrated.
Assessment Criteria: Judged on completeness and integration of skills. The project should solve a non-trivial problem and be demonstrably functional. Clarity of documentation and technical depth is critical.

Time Commitment: ~40 hours.

Checkpoint: End of Week 50 – Gather feedback (peer review or mentors). Identify any remaining gaps for future learning (to be carried into Weeks 51-52 if time).

Topic Comparison Table

Topic	Difficulty	Prerequisites	GenAI Career Impact
Agentic AI & Tools (MCP)	**Advanced**	LLM usage, API basics	**High** (enables intelligent agents) [1]
Prompt Engineering & CoT	Intermediate	LLM basics	High (foundation for reasoning tasks) [6]
Fine-tuning & Adaptation	**Advanced**	ML fundamentals, PyTorch	High (customize models for products) [8]
Retrieval-Augmented Gen (RAG)	Intermediate	Embeddings, databases	High (improves factual accuracy) [11]
Conversational Agents/ Chatbots	Intermediate	Bot frameworks, dialog flow	Medium-High (customer-facing apps)
Multimodal AI (Vision+Language)	Advanced	Vision basics, APIs	Medium (cutting-edge field) [16]
Azure OpenAI & AI Services	Intermediate	Azure basics	High (industry demand)
Deployment & MLOps	Advanced	DevOps, cloud services	High (production-readiness)
Evaluation & Metrics	Intermediate	Statistics	Medium (ensures quality)
Advanced Agents (Code Execution)	Advanced	Python, security	Medium-High (scalable agents) [2]
Responsible AI & Ethics	Intermediate	Awareness of AI issues	High (enterprise expectation)
LLM for Code Generation	Intermediate	Programming	Medium (developer tools)
Model Optimization (LoRA,Quant.)	Advanced	ML & hardware knowledge	Medium (cost savings)

Topic	Difficulty	Prerequisites	GenAI Career Impact
Domain-Specific GenAI	Intermediate	Domain knowledge	Medium-High (custom solutions)
Data Engineering for GenAI	Intermediate	Data pipelines	High (foundation of training)
LangChain & LLM Frameworks	Intermediate	Python, LLM API knowledge	High (accelerates development)
Cutting-edge Research & Trends	Advanced	Research literacy	Medium (innovation focus)
Capstone Project	Advanced	All of above	High (capstone showcases skills)

Checkpoints & Progress Reviews

At the end of each two-week slot, you should **review learning objectives and deliverables**. Ensure that the concrete deliverables (code, reports, demos) meet the assessment criteria. Example checkpoint questions:
- *Agentic AI & MCP:* Does your agent correctly use external tools? Can you add a new tool with minimal changes?
- *Prompt/CoT:* Do your prompts noticeably improve performance (e.g. higher accuracy)?
- *Fine-tuning:* Does the fine-tuned model outperform the base model on your domain?
- *RAG:* Are retrieved documents relevant? Does the RAG system produce factual answers?
- *Deployment:* Can you redeploy quickly via CI/CD? Does monitoring show correct behavior?

Use these reviews to decide if any topics need extra time. Document any issues and corrective actions before moving on.

3-Month Intensive Sprint Plan (Weeks 15–26)

For a focused 3-month sprint, we concentrate on the first six topics (Agentic AI through Azure Services) with a detailed weekly breakdown:

- **Week 15 (Mon–Sun):** *Agentic AI & Tools – Day 1–7*
- **Day 1:** Read Anthropic MCP announcement [1] and Google Apigee blog [3] (2h).
- **Day 2:** Follow an OpenAI function-calling tutorial (2h). Write Python script calling a sample API via GPT.
- **Day 3:** Code initial LangChain agent with a simple tool (web search) (4h).
- **Day 4–5:** Debug agent integration, refine prompts (4h).
- **Day 6:** Write README explaining tool usage (2h).
- **Day 7:** Self-review and checkpoint: test 3 sample queries, fix bugs (4h).
- **Week 16:** *Agentic AI & Tools – Day 8–14*
- **Day 8:** Learn MCP server basics (Anthropic docs) (2h).

• **Day 9:** Implement a local MCP server with two tools (3h).
• **Day 10:** Write MCP client in Python and connect (3h).
• **Day 11:** Test LLM calls to MCP tools; fix context prompts (4h).
• **Day 12:** Document MCP integration (2h).

• **Day 13–14:** Final testing and checkpoint (fix any context errors) (4h).

• **Week 17 (Prompt Engineering):** *Days 15–21*

• **Day 15:** Read Chain-of-Thought paper [6] (3h).
• **Day 16:** Implement CoT on sample math problems with GPT-4 (4h).
• **Day 17:** Compare few-shot vs zero-shot vs CoT (2h).
• **Day 18:** Build prompt templates for a task (e.g. summarization) (3h).
• **Day 19:** Test and refine prompts (2h).

• **Day 20–21:** Checkpoint: analyze how accuracy improved, prepare summary (4h).

• **Week 18:** *Days 22–28*

• **Day 22:** Experiment with self-consistency sampling (2h).
• **Day 23:** Tackle a mini-project (e.g. chatbot reasoning) (4h).
• **Day 24:** Optimize prompts with system messages (2h).
• **Day 25:** Write prompt guidelines document (2h).

• **Day 26–28:** Final tests and checkpoint (4h).

• **Week 19 (Fine-Tuning):** *Days 29–35*

• **Day 29:** Study Azure fine-tuning guide [8] (2h).
• **Day 30:** Collect or generate a small dataset; format as JSONL [9] (4h).
• **Day 31–32:** Run a quick fine-tune on a small model (4h).
• **Day 33:** Implement LoRA version (3h).
• **Day 34:** Evaluate outputs on test set (2h).

• **Day 35:** Checkpoint: document performance gains and token savings [7] (4h).

• **Week 20:** *Days 36–42*

• **Day 36–37:** Iterate training (tune hyperparams) (4h).
• **Day 38:** Deploy fine-tuned model to test (2h).
• **Day 39–40:** Compare outputs side-by-side; write report (4h).

• **Day 41–42:** Checkpoint and wrap-up (sample demos) (4h).

• **Week 21 (RAG):** *Days 43–49*

• **Day 43:** Read RAG paper [11] (3h).
• **Day 44:** Set up FAISS index with a text corpus (3h).

• **Day 45:** Query LLM with retrieval chain (3h).
• **Day 46:** Set up Azure Vector Search index [12] (4h).
• **Day 47:** Connect Azure OpenAI to query index (2h).
• **Day 48:** Test hybrid searches [13] (2h).

• **Day 49:** Checkpoint: compare RAG vs plain LLM answers (3h).

• **Week 22:** *Days 50–56*

• **Day 50–51:** Optimize retrieval (adjust k, embeddings) (3h).
• **Day 52:** Build simple UI or CLI for RAG QA (3h).
• **Day 53:** Document RAG pipeline with architecture diagram (2h).
• **Day 54–55:** Final tests, error-check (3h).

• **Day 56:** Checkpoint: ensure retrieval relevance and report accuracy gains (4h).

• **Week 23 (Conversational Agents):** *Days 57–63*

• **Day 57:** Set up Rasa or Bot Service skeleton (3h).
• **Day 58:** Create basic FAQ bot (no LLM) (3h).
• **Day 59:** Integrate LLM for response generation (3h).
• **Day 60:** Implement conversation memory (2h).
• **Day 61:** Test multi-turn dialogues (2h).
• **Day 62:** Add system prompts (2h).

• **Day 63:** Checkpoint: chat evaluation (4h).

• **Week 24:** *Days 64–70*

• **Day 64:** Wrap up conversational agent (fix dialog bugs) (3h).
• **Day 65:** Deploy chatbot (Azure Web App) (3h).
• **Day 66:** Conduct sample user session tests (3h).
• **Day 67:** Document bot logic and choices (2h).

• **Day 68–70:** Checkpoint: gather user feedback, finalize improvements (4h).

• **Week 25 (Multimodal AI):** *Days 71–77*

• **Day 71:** Read GPT-4 Vision report [16] (2h).
• **Day 72:** Use CLIP to match images and captions (3h).
• **Day 73:** Build image caption bot (API call or notebook) (3h).
• **Day 74:** Integrate GPT-4V (if available) for Q&A on images (3h).
• **Day 75:** Try DALL·E or Stable Diffusion for simple prompts (4h).

• **Day 76–77:** Checkpoint: analyze multimodal outputs, document capabilities (4h).

• **Week 26:** *Days 78–84*

- **Day 78:** Create a final multimodal demo (5h).
- **Day 79:** Test on varied inputs (3h).
- **Day 80:** Write short analysis of results (2h).
- **Day 81–84:** Sprint summary report (compilation of all learnings) (6h).

This sprint is intensive: plan for ~6–8 hours per day. Adjust as needed. Use weekends for catch-up or deep work.

Lightweight Alternative Plan

For a limited-time schedule, focus on **core priorities** and streamline work:

- **Top-Priority Topics:** Agentic AI, Prompt Engineering, RAG, Azure GenAI Services, Deployment, Ethics. (Skip deep dives into audio, video, advanced agents code, etc.)
- **Compressing Schedule:** Allocate 2–3 weeks per topic instead of 2. Use more prompts and fewer coding projects. For example:
- *Agents/MCP (Weeks 15–17):* Do a high-level project (LangChain agent) and read MCP summary.
- *Prompt/CoT (Weeks 18–19):* Focus on mastery of prompt patterns; skip building a full chatbot.
- *RAG (Weeks 20–21):* Set up a simple vector search + LLM demo (possibly using Azure Search directly).
- *Azure Deployment (Weeks 22–23):* Create one cloud project (e.g. deploy a basic web app calling Azure OpenAI).
- *Evaluations & Ethics (Weeks 24–25):* Review metrics and ethical guidelines at high level (maybe trade paper-reading for reading blog summaries).
- **Reduced Depth:** Use more "official quickstart" resources and prebuilt code examples. Focus on understanding rather than custom implementation.
- **Deliverables:** Aim for one solid project (e.g. a deployed chatbot with RAG) and concise write-ups instead of multiple mini-projects.

By concentrating on these key areas, even a 3–4 month plan can yield a strong foundational skillset in GenAI. Prioritize **hands-on practice** in Agents and RAG (high-impact) and ensure you at least touch on cloud integration and ethics.

Sources

We have prioritized official and academic sources throughout. For example, the Model Context Protocol (MCP) is detailed by Anthropic [1], and its efficient "code execution" approach is documented in Anthropic engineering blogs [2]. Chain-of-Thought prompting is validated by Wei et al. [6]. RAG was introduced by Lewis et al. (NeurIPS 2020) [11]. Microsoft's documentation provides guidance on fine-tuning (Azure Foundry) and LoRA [8] [7]. Azure's Vector Search overview explains using embeddings for RAG [12]. OpenAI's GPT-4 report confirms multimodal (vision+language) capabilities [16]. These and other references are cited inline and can be consulted for deeper details on each topic.

---

[1] Introducing the Model Context Protocol \ Anthropic
https://www.anthropic.com/news/model-context-protocol

[2] Code execution with MCP: building more efficient AI agents \ Anthropic
https://www.anthropic.com/engineering/code-execution-with-mcp

3 The agentic experience: Is MCP the right tool for your AI future? - Google Developers Blog
https://developers.googleblog.com/the-agentic-experience-is-mcp-the-right-tool-for-your-ai-future/

4 5 20 Agents - Docs by LangChain
https://docs.langchain.com/oss/javascript/langchain/agents

6 [2201.11903] Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
https://arxiv.org/abs/2201.11903

7 8 9 18 Customize a model with fine-tuning - Microsoft Foundry | Microsoft Learn
https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/fine-tuning

10 Fine-tuning · Hugging Face
https://huggingface.co/docs/transformers/en/training

11 [2005.11401] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
https://arxiv.org/abs/2005.11401

12 13 Vector Search Overview - Azure AI Search | Microsoft Learn
https://learn.microsoft.com/en-us/azure/search/vector-search-overview

14 Building Your Own Chatbot using Azure OpenAI Capabilities
https://techcommunity.microsoft.com/blog/educatordeveloperblog/building-your-own-chatbot-using-azure-openai-capabilities/4260740

15 Building a Basic Chatbot with Azure OpenAI
https://techcommunity.microsoft.com/blog/educatordeveloperblog/building-a-basic-chatbot-with-azure-openai/4373018

16 17 cdn.openai.com
https://cdn.openai.com/papers/gpt-4.pdf

19 Safety & responsibility - OpenAI
https://openai.com/safety/